ロール追加ダイアログUI設計書

メタデータ

title: ロール追加ダイアログUI設計書
category: UI設計
version: 1.0.0
last_updated: 2026-02-02
author: Development Team
tags: [system-admin, role-management, dialog, add-form]

関連ドキュメント

- ロール管理画面
- ロール編集ダイアログ
- 機能管理ダイアログ
- ロール管理API仕様
- UIポリシー

---

1. 概要

1.1. 目的

システム管理者が新規ロールを追加するためのモーダルダイアログです。ロールCD、ロール名、機能、初期画面機能名、状態を入力して登録します。

1.2. 表示条件

- ロール管理画面の「追加」ボタンをクリック

- System Admin権限が必要

1.3. 画面イメージ

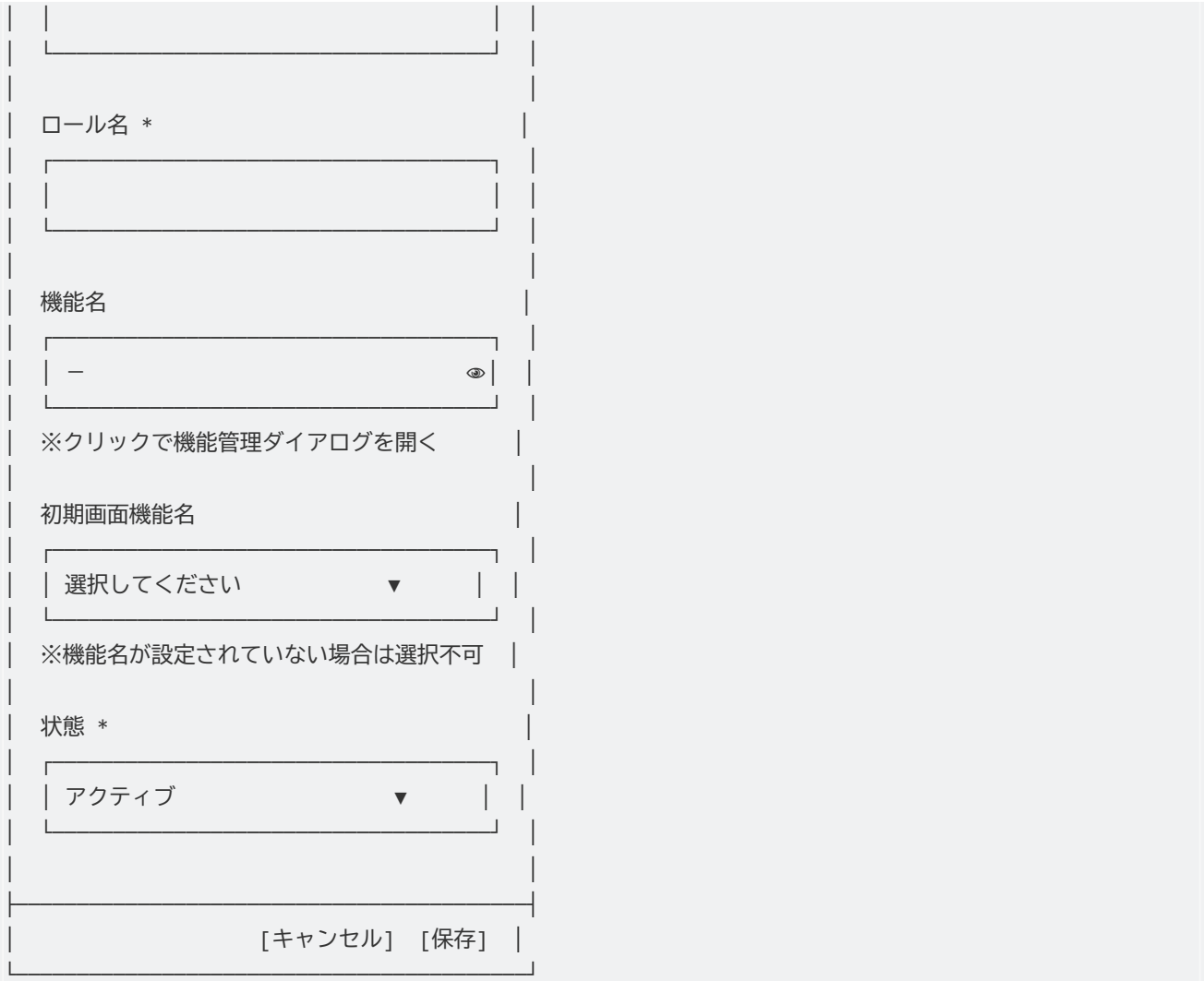


---

2. ダイアログ構成

2.1. レイアウト

```
┌──────────────────────────────────────┐
│ ロール追加                      [×]  │
├──────────────────────────────────────┤
│                                      │
│  ロールCD *                          │
│  ┌──────────────────────────────┐    │
```



2.2. モーダル仕様

コンポーネント: `<Modal>`

仕様:

項目	値
タイトル	「ロール追加」
幅	600px
位置	画面中央
マスク	背景を暗くする
キーボードESC	閉じる（確認なし）
マスククリック	閉じない

---

3. 入力項目

3.1. ロールCD（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「ロールCD」
必須	○
プレースホルダー	「ロールCDを入力してください」
最大文字数	50文字
許可文字	英数字、アンダースコア
重複チェック	あり

バリデーション:

```
{
  required: true,
  message: 'ロールCDを入力してください'
},
{
  max: 50,
  message: 'ロールCDは50文字以内で入力してください'
},
{
  pattern: /^[a-zA-Z0-9_]+$/,
  message: 'ロールCDは英数字とアンダースコアのみ使用できます'
}
```

重複チェック:

- API呼び出し時に重複をチェック
- 重複の場合: `"入力されたIDは既に存在しています"` エラー
- フォームフィールドにエラーを表示

3.2. ロール名（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「ロール名」
必須	○
プレースホルダー	「ロール名を入力してください」
最大文字数	100文字

バリデーション:

```
{
  required: true,
  message: 'ロール名を入力してください'
},
{
  max: 100,
  message: 'ロール名は100文字以内で入力してください'
}
```

3.3. 機能名（読み取り専用、クリック可能）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「機能名」
必須	×
初期値	「−」
読み取り専用	○
クリック可能	○
背景色	`token.colorBgContainer`

カーソル	`pointer`

動作:

1. フィールドをクリック
2. 機能管理ダイアログを開く（temporaryモード）
3. 機能を選択して「機能を更新」をクリック
4. 機能名がメモリに一時保存される
5. フォームフィールドに反映される（例: `Ⓦシステム管理者メニュー`）
6. 初期画面機能名フィールドが活性化される

注意事項:

- ロールCDが未入力でもダイアログは開ける
- 機能選択はメモリに一時保存され、「保存」ボタンクリック時にDBに反映

実装例:

```
<Form.Item
  name="functionName"
  label="機能名"
>
  <Input
    readOnly={true}
    placeholder="ー"
    onClick={() => {
      // ロールCDが未入力でもダイアログを開く
      const tempMaster = {
        role: form.getFieldValue('role') || '',
        roleName: form.getFieldValue('roleName') || '',
        functionName: 'ー',
        functionWritePermissions: '',
        functionIds: [],
        writableFunctionIds: [],
        isActive: true,
      };
      handleOpenFunctionModalTemporary(tempMaster);
    }}
    style={{
      cursor: 'pointer',
      backgroundColor: token.colorBgContainer
    }}
```

```
  />
</Form.Item>
```

3.4. 初期画面機能名（条件付きSelect）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「初期画面機能名」
必須	×
プレースホルダー	「選択してください」
無効化条件	機能名が未設定または「−」の場合
クリア可能	○

選択肢:

- 機能名フィールドで選択した機能のリスト
- 権限アイコン（ⓌとⓇ）を除去した機能名を表示
- 複数機能の場合、それぞれが選択肢になる

例:

- 機能名が `Ⓦシステム管理者メニュー / Ⓡ実験管理者メニュー` の場合
- 選択肢:
  - 「システム管理者メニュー」
  - 「実験管理者メニュー」

動的な活性/非活性制御:

```
<Form.Item
  noStyle
  shouldUpdate={(prevValues, currentValues) =>
    prevValues.functionName !== currentValues.functionName
  }
>
```

```
  {({ getFieldValue }) => {
    const functionNameValue = getFieldValue('functionName');
    const isFunctionDisabled = !functionNameValue || functionNameValue === '—';

    return (
      <Form.Item
        name="initialScreenFunctionName"
        label="初期画面機能名"
      >
        <Select
          placeholder="選択してください"
          disabled={isFunctionDisabled}
          options={functionOptions}
          allowClear
        />
      </Form.Item>
    );
  }}
</Form.Item>
```

3.5. 状態（必須）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「状態」
必須	○（デフォルト値あり）
初期値	`true`（アクティブ）

選択肢:

```
[
  { value: true, label: 'アクティブ' },
  { value: false, label: '非アクティブ' }
]
```

---

4. アクションボタン

4.1. 配置

位置: モーダルフッター（右寄せ）

レイアウト:

```
<Space>
  <Button onClick={handleModalCancel}>キャンセル</Button>
  <Button type="primary" onClick={handleModalOk}>保存</Button>
</Space>
```

4.2. キャンセルボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「キャンセル」
タイプ	`default`

動作:

1. モーダルを閉じる
2. フォームをリセット
3. メモリに保存された機能情報をクリア
4. 確認なし

4.3. 保存ボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「保存」
タイプ	`primary`
ローディング	送信中は `true`

動作フロー:

1. フォームバリデーション実行
2. バリデーションエラーがある場合、エラーメッセージ表示
3. ローディング状態に変更
4. **Step 1**: ロールテーブルに新規追加（`createRole` API）
5. **Step 2**: メモリに機能情報がある場合、権限を登録（`updateRolePermissions` API）
6. 成功時:
   - メモリをクリア
   - モーダルを閉じる
   - データ再読み込み
   - 成功メッセージ表示: `"ロールを追加しました"`
7. 失敗時:
   - エラーメッセージ表示
   - モーダルは開いたまま

実装例:

```
const handleModalOk = async () => {
  try {
    const values = await form.validateFields();

    // 1. ロールテーブルに追加
    await createRole({
      role: values.role,
      roleName: values.roleName,
      initialScreenFunctionName: values.initialScreenFunctionName || null,
      isActive: values.isActive !== undefined ? values.isActive : true,
    });

    // 2. メモリに保存された機能情報があれば権限も登録
```

```
    if (tempFunctionData && tempFunctionData.functionName !== 'ー') {
      await updateRolePermissions(
        values.role,
        tempFunctionData.functionName,
        tempFunctionData.functionWritePermissions
      );
    }

    await fetchRoleMasters();
    message.success('ロールを追加しました');

    // メモリをクリア
    setTempFunctionData(null);

    setIsModalVisible(false);
    form.resetFields();
  } catch (error) {
    console.error('エラー:', error);
    const errorMessage = error instanceof Error ? error.message : 'エラーが発生しました';

    if (errorMessage.includes('既に存在しています')) {
      form.setFields([
        {
          name: 'role',
          errors: [errorMessage],
        },
      ]);
      message.error(errorMessage);
      return;
    }

    message.error(errorMessage);
  }
};
```

---

5. API仕様

5.1. ロール新規作成

エンドポイント: `POST /api/v1/systemManagement/role`

リクエストヘッダー:

```
Content-Type: application/json
Authorization: Bearer {session_token}
```

リクエストボディ:

```
{
  "role": "mi_engineer",
  "roleName": "MI技術者",
  "initialScreenFunctionName": null,
  "isActive": true
}
```

レスポンス（成功時）:

```
{
  "success": true,
  "data": {
    "roleId": "mi_engineer",
    "roleName": "MI技術者",
    "initialScreenFunctionName": null,
    "isActive": true,
    "createdAt": "2026-02-02T09:00:00.000Z"
  },
  "meta": {
    "timestamp": "2026-02-02T09:00:00.000Z",
    "requestId": "req_abc123"
  }
}
```

レスポンス（エラー時 - 重複）:

```
{
  "success": false,
  "error": {
    "code": "RESOURCE_CONFLICT",
    "message": "入力されたIDは既に存在しています"
  },
  "meta": {
    "timestamp": "2026-02-02T09:00:00.000Z",
    "requestId": "req_abc123"
```

```
  }
}
```

5.2. ロール権限登録

エンドポイント: `PUT /api/v1/systemManagement/role/:roleId/permissions`

リクエストヘッダー:

```
Content-Type: application/json
Authorization: Bearer {session_token}
```

リクエストボディ:

```
{
  "functionName": "Ⓦシステム管理者メニュー / Ⓡ実験管理者メニュー",
  "functionWritePermissions": "システム管理者メニュー"
}
```

レスポンス（成功時）:

```
{
  "success": true,
  "meta": {
    "timestamp": "2026-02-02T09:00:00.000Z",
    "requestId": "req_abc123"
  }
}
```

---

6. バリデーション

6.1. クライアントサイドバリデーション

必須チェック:

- ロールCD
- ロール名
- 状態

形式チェック:

- ロールCD: 英数字とアンダースコアのみ
- 最大文字数: ロールCD（50文字）、ロール名（100文字）

6.2. サーバーサイドバリデーション

重複チェック:

- ロールCDが既存のロールと重複していないか

データ整合性チェック:

- 初期画面機能名が選択された機能のリストに含まれているか

---

7. エラーハンドリング

7.1. エラーメッセージ一覧

エラーコード	メッセージ	表示場所
`VALIDATION_ERROR`	「ロールCDを入力してください」	フォームフィールド
`VALIDATION_ERROR`	「ロール名を入力してください」	フォームフィールド
`VALIDATION_ERROR`	「ロールCDは英数字とアンダースコアのみ使用できます」	フォームフィールド
`RESOURCE_CONFLICT`	「入力されたIDは既に存在しています」	トースト + フォームフィールド
`SYSTEM_ERROR`	「エラーが発生しました」	トースト

7.2. エラー表示方法

フォームフィールドエラー:

```
form.setFields([
  {
    name: 'role',
    errors: ['入力されたIDは既に存在しています'],
  },
]);
```

トーストメッセージ:

```
message.error('入力されたIDは既に存在しています');
```

---

8. ユーザーフロー

8.1. 正常フロー

```
[ロール管理画面]
    ↓ 「追加」ボタンクリック
[ロール追加ダイアログ] 表示
    ↓ ロールCD入力: "mi_engineer"
    ↓ ロール名入力: "MI技術者"
    ↓ 機能名フィールドクリック
[機能管理ダイアログ] 表示（temporaryモード）
    ↓ 機能選択: システム管理者メニュー（書込可）
    ↓ 「機能を更新」クリック
[ロール追加ダイアログ] に戻る
    機能名: "Ⓦシステム管理者メニュー"（メモリに保存）
    初期画面機能名: 活性化され、「システム管理者メニュー」を選択可能
    ↓ 初期画面機能名選択（オプション）
    ↓ 状態選択: アクティブ
    ↓ 「保存」ボタンクリック
API呼び出し: createRole()
API呼び出し: updateRolePermissions()（機能が設定されている場合）
    ↓ 成功
[ロール管理画面]
    データ再読み込み
    成功メッセージ表示: "ロールを追加しました"
```

8.2. エラーフロー（重複）

```
[ロール追加ダイアログ]
    ↓ ロールCD入力: "provider"（既存のロールCD）
    ↓ ロール名入力: "プロバイダー"
    ↓ 「保存」ボタンクリック
API呼び出し: createRole()
    ↓ エラー（409 Conflict）
[ロール追加ダイアログ]
    エラーメッセージ表示: "入力されたIDは既に存在しています"
    ロールCDフィールドにエラー表示
    モーダルは開いたまま
    ↓ ロールCDを修正
    ↓ 再度「保存」ボタンクリック
```

---

9. アクセシビリティ

9.1. キーボード操作

Tab順序:

1. ロールCD入力
2. ロール名入力
3. 機能名入力（クリック可能）
4. 初期画面機能名選択
5. 状態選択
6. キャンセルボタン
7. 保存ボタン

ショートカットキー:

- Enter: 保存ボタンのクリック（フォーカスが入力フィールドにない場合）
- Escape: モーダルを閉じる

9.2. ARIA属性

```
<Modal
  title="ロール追加"
  open={isModalVisible}
  onCancel={handleModalCancel}
  aria-labelledby="role-add-dialog-title"
  aria-describedby="role-add-dialog-description"
>
  <Form form={form} layout="vertical">
    <Form.Item
      label="ロールCD"
      name="role"
      rules={[{ required: true }]}
    >
      <Input
        aria-label="ロールCD"
        aria-required="true"
      />
    </Form.Item>
  </Form>
</Modal>
```

---

10. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-02-02	初版作成	Development Team